Exceptions

C++ Interlude 3

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Background

- Sometimes it is not possible for a method to return a boolean value when an unusual situation prevents it from completing its task
- Preconditions for a method not always met
  - Method might return a false to indicate this
  - But not always possible
- Example
  - Stack method peek() called on empty stack which contains items of type bool
  - Return cannot be sure if return is normal or an exception

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Assertions

- Express an assertion either as a comment or by using the C++ function assert
  - #include <cassert>
  - Make assertions about variables, objects
  - Assertion in form of boolean expression that should be true at that point in program
  - False halts program execution
- Mainly used to validate pre- or post-conditions
- This is a debugging tool
  - Not a substitute for an if statement
  - Should not be deployed in final program

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Assertions

```
#include <cassert>
#include <iostream>

int main()
{
   std::cout << "Hello world 1" << std::endl;
   int x = 7;

   /*  Some big code in between and let's say x
     is accidentally changed to 9  */
   x = 9;

   std::cout << "Hello world 2" << std::endl;

   // Programmer assumes x to be 7 in rest of the code
   assert(x==7);

   /* Rest of the code */
    std::cout << "Hello world, again" <<std::endl;
    return 0;
}
```

- Assertions
  - can be useful when you are testing and debugging your program.
  - is not be the best solution for a final shipping program.
- A violation of a function's preconditions may be a simple mistake that the client can fix.
- Such a situation does not warrant terminating program execution.

What is the console output?

Hello world 1
Hello world 2
Assertion failed: x == 7, …

Program halts.
This is not executed.

Throwing Exceptions

- Alternate way of communicating or returning information to function's client
- Thrown exception bypasses normal execution,
  - Control immediately returns to client.
- Can contain information about the error or unusual condition that helps you resolve the issue and possibly try the function again

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Throwing Exceptions

Syntax

```
throw ExceptionClass(stringArgument);
```

logic_error exceptions (or one of its derived classes) represent errors in program logic (e.g., unmet precondition or failure to satisfy a postcondition, that you could detect before the program executes).

- test for an error or unusual condition using an if statement; throw an exception containing a message to our client, if error occurs
- To restrict the exceptions, you can include a throw clause in its header, informing you that the function might throw an exception.
- Including a throw clause in a function specification ensures that the function can throw only those exceptions. An attempt by the function to throw any other exception will result in a runtime error.
- Omitting a throw clause allows a function to throw any exception
- You can handle only exceptions of the class logic_error and its derived classes. To handle an exception, you write code that consists of two pieces:
    - **try** block - contains the statements that might cause or throw an exception
    - one or more **catch** blocks that follow the try block

runtime_error exceptions (or one of its derived classes) are thrown by errors that are detected at runtime (e.g., division by zero or numeric overflow)

- program will halt, just like an assertion fail
- can contain a message that is displayed to the user giving details about the error and the reason the program halted

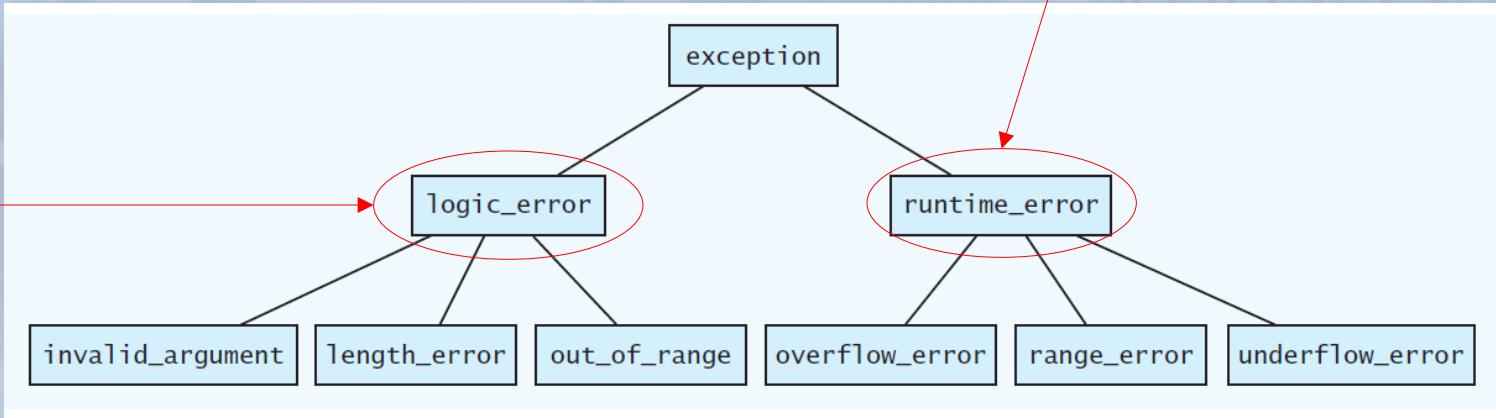

FIGURE C3-1 Hierarchy of C++ exception classes

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Throwing Exceptions

```
PlainBox<std::string> findBox(PlainBox<std::string> boxes[], int size,
                              std::string target) throw(std::logic_error)
{
   int index = 0;
   bool found = false;
   while (!found && (index < size))
   {
      found = (target == boxes[index].getItem());
      if (!found)
         index++; // Look at next entry
   }  // end while

   if (!found)
      throw std::logic_error("Target not found in a box!");
   return boxes[index];
}  // end findBox
```

- If you can resolve the unusual situation in a reasonable manner, your code should do so instead of throwing an exception.
- If several resolutions to an abnormal occurrence are possible, and you want the client to choose one, you **could**/should throw an exception.
- If you detect that a programmer has used your method incorrectly, you can throw a runtime exception. Recall that this action will halt program execution.

LISTING C3-3 Revised findBox function that throws an exception

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Handling Exceptions

- Code for handling exception
  - try block – contains statements that might cause or throw an exception
  - catch block – immediately follows try block with code to react to or catch a particular type of exception

```
try
{
    < statement(s) that might throw an exception >
}
catch (ExceptionClass  identifier)
{
    < statement(s) that react to an exception of type ExceptionClass >
}
```

General syntax for a try block followed by one catch block

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Handling Exceptions

- try block
  - Contains statements that might cause or throw an exception
- catch block
  - One or more catch blocks immediately follow try block
  - Contain code to react to or catch particular type of exception

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Handling Exceptions

- If no exception occurs and try block completes
  - Execution continues with statement after catch block
- If statement within try block causes exception of type specified in catch block
  - Remainder of try block abandoned
  - Execution transfers to statements in catch block
  - After catch block statements finish, execution jumps to statement after last catch block

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Handling Exceptions

- The syntax for catch block resembles that of a function definition
  - Specifies type of exception, and an identifier
  - The catch block parameter provides name for caught exception
- Steps taken in catch block vary
  - Simple message
  - Elaborate update of variables, retry of offending function

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Handling Exceptions

LISTING C3-4 Trying the function `findBox`

```
// Create and initialize an array of boxes
PlainBox<string> myBoxes[5];          // Array of PlainBox objects
myBoxes[0] = PlainBox<string>("ring");
myBoxes[1] = PlainBox<string>("hat");
myBoxes[2] = PlainBox<string>("shirt");
myBoxes[3] = PlainBox<string>("sock");
myBoxes[4] = PlainBox<string>("shoe");
PlainBox<string> foundBox;
```

```
// Try to find a box containing glasses
try
{
   foundBox = findBox(myBoxes, 5, "glasses");
}
catch(logic_error logErr)
{
   cout << logErr.what() << endl;          // Display error message to user
   foundBox = PlainBox<string>("nothing"); // Fix problem
}  // end try-catch
// Because we catch the exception and fix the problem, the following
// statement should work even if the target is not found
cout << foundBox.getItem();
```

Output

```
Target not found in a box!
nothing
```

LISTING C3-4 Trying the function findBox

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Multiple catch Blocks

- try block may cause more than one type of exception
  - Can have many catch blocks associated with it
- catch blocks must be ordered
  - Most specific classes first
  - More general classes last

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Multiple catch Blocks

```
string str = "Sarah";
try
{
    str.substr(99, 1);
    < other statements appear here . . . >
}
catch (exception e)            // WRONG ORDER!!
{
    cout << "Something else was caught" << endl;
}
catch (out_of_range e)
{
    cout << "out_of_range exception caught" << endl;
}   // end try-catch
```

The compiler may issue a warning message similar to the following one:

```
'class std::out_of_range' : is caught by base class ('class  exception')
on line n
```

- Arrange catch blocks in order of specificity, catching the most specific one first.
- Since all exception classes have exception as an ancestor, try to avoid using exception in a catch block.
- If you must catch exceptions of type exception , do so in the last catch block.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Uncaught Exceptions

- When an exception can occur within a loop, you must decide whether to:
  - place a try block within the body of the loop, in which case an exception will be caught each time the loop cycles or
  - place the entire loop within a try block—in which case the exception would be caught only once
- Neither choice is better than the other, but if you make the wrong choice, the client likely will not get the necessary information.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Programmer-Defined Exception Classes

- There is a C++ Standard Library exception class defined.
- You may also want to define your own exception class.
- Usually, C++ exception class exception , or one of its derived classes, is the base class for programmer defined exception classes
  - Provides a standardized interface for working with exceptions.
- Exception class typically consists of a constructor that has a string parameter

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Programmer-Defined Exception Classes

```
#include <stdexcept>
#include <string>
class TargetNotFoundException: public std::exception
{
public :
   TargetNotFoundException(const std::string& message = "")
                          : std::exception("Target not found: " + message)
   {
   } // end constructor
}; // end TargetNotFoundException
```

```
throw TargetNotFoundException (target + " not found in a box!");
```

Example – constructor provides way for throw statement to identify condition of exception.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Exceptions - Summary

- To throw an exception in a function:
  - Determine what errors, preconditions, postconditions, and unusual conditions you must check and where they should be checked, so your function can correctly perform its task. Usually this is done with an if statement.
  - If an error condition has occurred, throw an exception using the throw statement. As an argument to the exception class constructor, pass a string description of the reason for the error.
  - Add a throw clause to the function header indicating the exceptions thrown by the function.
  - Derive a custom exception class to better identify the conditions that caused the error, though doing so is optional, it is desirable.
- To handle an exception or use a function that could throw an exception
  - Place the statement that might throw an exception in a try block. The fewer statements in the try block the better.
  - Do not use new to create an object in a try block.
  - After the try block, place catch blocks for each type of exception that can be thrown by the statements in the try block.
  - Place the catch blocks in order from the most specific exception classes to the more general exception classes.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

End

C++ Interlude 3

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Appendix

C++ Interlude 3

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Problem to Solve

- Previous C++ Interlude worked on video game
- Next task
  - Create function that searches for given string in a number of boxes
- Function parameters
  - Array of string objects
  - Integer represents number of objects in array
  - String to be located

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Problem to Solve

```
PlainBox<std::string> findBox(PlainBox<std::string> boxes[], int size,
                              std::string target)
{
   int index = 0;
   bool found = false;
   while (!found && (index < size))
   {
      found = (target == boxes[index].getItem());
      if (!found)
         index++; // Look at next entry
   }  // end while
   return boxes[index];
}  // end findBox
```

LISTING C3-1 First try at the function findBox

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Problem to Solve

- Must deal with problem of a box containing target string not in the array
  - If target not found, function returns boxes[size] which is undefined
  - Problems occur when client tries to use this “box”
- What to return when target not found?

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Assertions

```
PlainBox<std::string> findBox(PlainBox<std::string> boxes[], int size,
                              std::string target)
{
   int index = 0;
   bool found = false;
   while (!found && (index < size))
   {
      found = (target == boxes[index].getItem());
      if (!found)
         index++;   // Look at next entry
   }  // end while
   assert(found);   // Verify that there is a box to return
   return boxes[index];
}  // end findBox
```

LISTING C3-2 Revised findBox function with assertions

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Uncaught Exceptions

```
#include <iostream>
#include <string>


// Encodes the character at index i of the string str.
void encodeChar(int i, string& str)
{
    int base = static_cast<int>('a');
    if (isupper(str[i]))
       base = int('A');

    char newChar = (static_cast<int>(str[i]) - base + 3) % 26 + base;
    str.replace(i, 1, 1, newChar); // Method replace can throw exception
}   // end encodeChar

// Encodes numChar characters within a string.
void encodeString(int numChar, string& str)
```

LISTING C3-5 A program with an uncaught exception

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Uncaught Exceptions

```
// Encodes numChar characters within a string.
void encodeString(int numChar, string& str)
{
   for (int j = numChar - 1; j >= 0; j-)
      encodeChar(j, str);
}  // end encodeString

int main()
{
    string str1 = "Sarah";
    encodeString(99, str1);
    return 0;
}   // end main
```

LISTING C3-5 A program with an uncaught exception

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Uncaught Exceptions

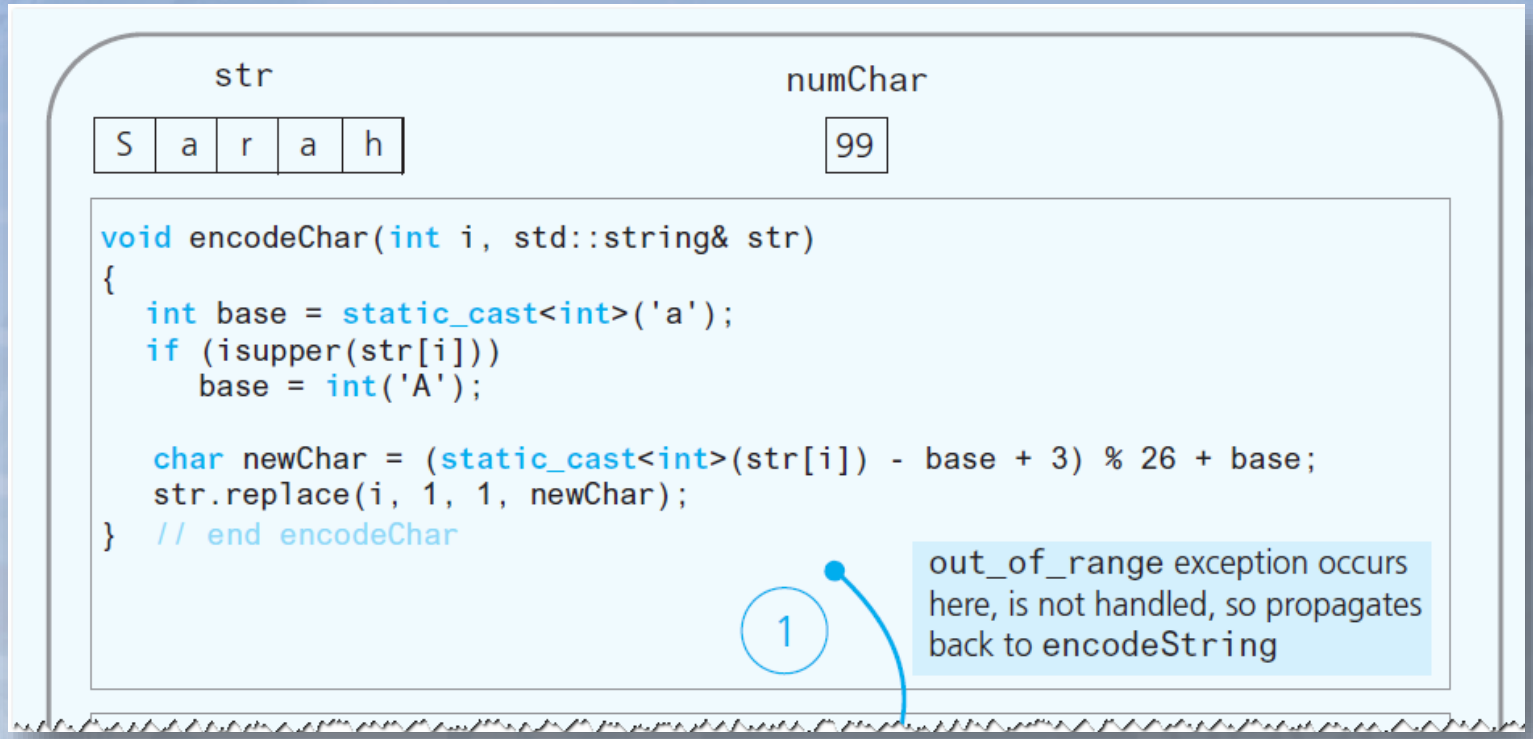

FIGURE C3-2 Flow of control for an uncaught exception

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Uncaught Exceptions

```
void encodeString(int numChar, std::string& str)
{
   for (int j = numChar-1; j >= 0; j--)
      encodeChar(j, str);
}  // end encodeString
```

2

out_of_range exception not handled here, so propagates back to main

The function main

```
int main()
{
   std::string str1 = "Sarah";
   encodeString(99,  str1);
   return 0;
}  // end main
```

3

out_of_range exception not handled in main; causes abnormal program termination

Output:

```
abnormal program termination
```

FIGURE C3-2 Flow of control for an uncaught exception

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Handling Exceptions

```
try
{
    < statement(s) that might throw an exception >
}
catch (ExceptionClass  identifier)
{
    < statement(s) that react to an exception of type ExceptionClass >
}
```

General syntax for a try block followed by one catch block

© 2017 Pearson Education, Hoboken, NJ. All rights reserved